```
        w.WriteHeader(http.StatusInternalServerError)
        return
    }
    fmt.Fprintf(w, message)
}

func main() {
    rand.Seed(time.Now().UnixNano())
    http.HandleFunc("/echo", handleEcho)
    log.Fatal(http.ListenAndServe(":4242", nil))
}
```

Return the exact payload we retrieved in 90% of the cases

Register handler for the /echo path

Launches the HTTP server that will serve on port 4242 on all available network interfaces

First, let's build the `echo` service, as shown in the following listing.

Listing 2.2 Building our example Go program

```
$ go build -o echosvc .
```

Builds the executable, which is then available in the current directory as ./echosvc

Now, you can run the service with `./echosvc` in the same directory. To invoke it via its HTTP API, you would use the following code (in a separate terminal session):

```
$ curl localhost:4242/echo?message=test

test
```

By default, I will use Grafana as the destination, since it allows us to visualize and interact with a wide array of data sources in a uniform manner. This is will allow you to reproduce everything for free with open source. Now that we've laid out our reference example, let's talk about a way to assess the costs of instrumentation, before we move to the instrument itself.

Tip

To see the signals going forward, in Grafana, you will need to put some load onto the `echo` service. One option is to do something like the following directly from the shell (calling the endpoint every second):

```
while true ; do curl -s -o
/dev/null localhost:4242/echo ; sleep 1 ; done
```

Alternatively, you can use something more fancy, such as a full-blown testing tool like Salvo (https://github.com/tarekziade/salvo) or even K6 (https://k6.io/docs/testing-guides/api-load-testing/).

2.2 Assessing instrumentation costs

To assess the costs of instrumentation per signal type, we will use a simple measure: the *business logic to instrumentation* (B2I) ratio.